ENSIM
École d'ingénieurs
Le Mans Université



Projet 3A

Flot robotique

Professeur :
Kais HASSAN

Équipe :
Nolan DELANNÉE
Nolhan BERTAULT
Mathéo BIDEAULT
Ilyes HASNI

2025/2026

Le Mans Université

Table des matières

1 Introduction **3**

2 Présentation du robot **4**
2.1 Robofleet 4
2.2 Roue Mécanum 5
2.3 Le LiDAR 6
2.4 Jetson Orin Nano 7

3 Architecture logicielle **9**
3.1 Choix de l'environnement logiciel 9
3.2 Organisation en packages 9
3.3 Description des packages 10
3.3.1 turn_on_wheeltec_robot — Initialisation du robot 10
3.3.2 nav2_bringup — Navigation et cartographie 10
3.3.3 wheeltec_multi — Coordination maître-esclave 11
3.3.4 rosbridge_server — Interface avec Foxglove 11
3.3.5 wheeltec_robot_keyboard — Contrôle clavier 11

4 Réalisation et résultats **12**
4.1 Fonctionnalités réalisées 12
4.2 Difficultés rencontrées et solutions apportées 12
4.2.1 Optimisation de la communication maître-esclave 12
4.2.2 Récupération des fichiers de carte 12

5 Conclusion **13**

Bibliographie **14**

Table des figures

2.1 Robofleet 4
2.2 La roue mécanum 5
2.3 Orientation de la roue 6
2.4 LiDAR M10 6
2.5 Principe de fonctionnement du LiDAR par temps de vol 7
2.6 NVIDIA Jetson Orin Nano 7

4.1 Carte 12

Introduction

Le projet

Dans le cadre de notre formation en troisième année à l'ENSIM, un projet nous a été attribué et occupera une grande partie celle-ci. Son objectif est de nous apprendre à gérer un projet en équipe, car, dans notre future vie d'ingénieur, la plupart des projets s'effectuent à plusieurs, il est donc nécessaire d'apprendre à s'organiser en équipe.

Notre projet s'intitule flot robotique le principe est qu'on dispose de trois robot identique capable de communiquer entre eu et l'objectif est que deux d'entre eu les "esclaves" avance en suivant un robot "maître". Chaque robot possède différents capteur et peut être contrôler. L'objectif final sera de faire une démonstration des différentes possibilités de la flot robotique et de fournir un document technique qui facilitera la prise en mains pour les prochains utilisateur.

Ce compte rendu est composé de plusieurs parties. Tout d'abord, nous présenterons le robot ainsi que l'ensemble de ses caractéristiques techniques. Ensuite, nous décrirons l'architecture logicielle du système, c'est-à-dire l'organisation des programmes, leurs interactions et les choix de conception qui ont guidé notre travail. Dans une troisième partie, nous présenterons la réalisation concrète du projet : les fonctionnalités mises en œuvre, les difficultés rencontrées et les solutions apportées. Enfin, nous conclurons en donnant notre ressenti, à la fois individuel et collectif, sur le projet, avant d'aborder les perspectives d'avenir.

L'équipe

Notre équipe est constituée de :
— DELANNÉE Nolan(Chef de Projet)
— BERTAULT Nolhan
— BIDEAULT Mathéo
— HASNI Ilyes

Présentation du robot

2.1 Robofleet

Le Robofleet est un système de robots mobiles conçu par la société Roboworks. Il s'agit d'un ensemble de robots identiques destinés à la recherche et à l'enseignement en robotique multi-agents.

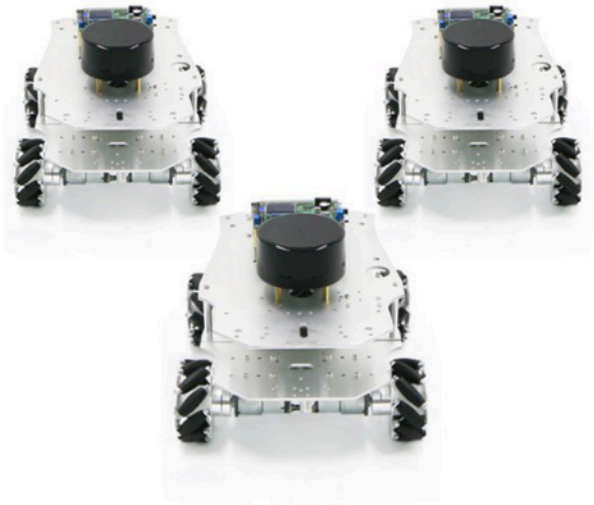


FIGURE 2.1 – Robofleet

Le **Robofleet** est commercialisé sous forme de packs comprenant 3, 5, 10 ou 15 robots. Tous les robots sont strictement identiques, tant du point de vue matériel que logiciel. L'ensemble du système peut créer un réseau Wi-Fi local dédié, un routeur étant inclus dans le package. Chaque robot est capable de fonctionner de manière autonome, mais également de collaborer avec les autres. Le système repose sur une architecture embarquée basée sur une carte NVIDIA Jetson Orin Nano (4 Go), qui assure les capacités de calcul. Les robots fonctionnent sous le système d'exploitation Ubuntu 22.04

et utilisent le middleware robotique ROS 2 Humble, largement répandu en recherche et développement pour la gestion des communications, de la perception et du contrôle.

Le Robofleet est équipé d'un LiDAR LeiShen M10P offrant une vision à 360° avec une portée de 30 m, permettant la cartographie, la détection d'obstacles et la navigation autonome. Le déplacement est assuré par des roues mécanum, offrant une mobilité omnidirectionnelle adaptée à la coordination de plusieurs robots. Le système intègre également des algorithmes de formation, l'évitement d'obstacles, ainsi que des packages ROS et des applications mobiles de contrôle. Enfin, les robots communiquent via un réseau Wi-Fi local permettant l'échange de données en temps réel et la coordination de la flotte.

2.2 Roue Mécanum

La **roue Mecanum** est une roue omnidirectionnelle inventée en 1973 par Bengt Ilon. Elle permet au robot de se déplacer non seulement vers l'avant, l'arrière ou en rotation, mais aussi latéralement et en diagonale, sans changer l'orientation du châssis.

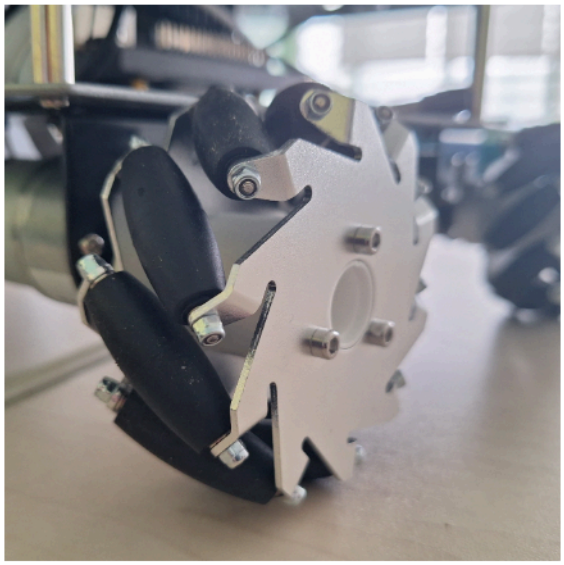


FIGURE 2.2 – La roue mécanum

Cette capacité est rendue possible grâce aux petits rouleaux inclinés à 45° placés autour de la roue. En combinant la rotation des différentes roues, le robot peut effectuer des mouvements précis dans toutes les directions.

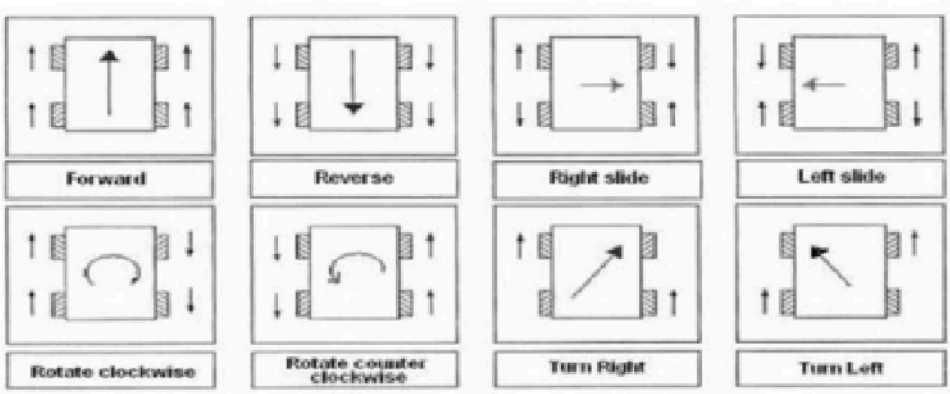


FIGURE 2.3 – Orientation de la roue

Les roues Mecanum sont particulièrement utiles dans les espaces restreints et sont utilisées dans de nombreux domaines comme la logistique, la robotique ou les compétitions robotiques.

2.3 Le LiDAR

Le LiDAR (Light Detection And Ranging) est un dispositif optoélectronique de télédétection qui émet des impulsions laser et mesure le temps de vol du signal réfléchi afin de déterminer la distance des objets environnants. Le modèle utilisé dans le Robofleet est le LiDAR M10, capable d'assurer une vision à 360° avec une portée d'environ 30 mètres, ce qui permet la cartographie de l'environnement, la détection d'obstacles et la navigation autonome.



FIGURE 2.4 – LiDAR M10

Le fonctionnement du LiDAR repose sur le principe du temps de vol (Time of Flight, ToF). Le capteur mesure le temps nécessaire à une impulsion laser pour atteindre un obstacle puis revenir vers le capteur.

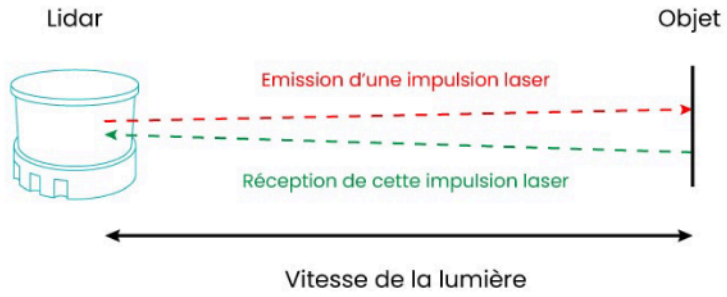


FIGURE 2.5 – Principe de fonctionnement du LiDAR par temps de vol

La distance est alors calculée à l'aide de la relation :

$$t_{\mathrm{ToF}} = \frac{2R}{c}$$

avec :
— R : distance entre le capteur et l'obstacle ;
— c : vitesse de la lumière.
La distance peut ensuite être obtenue par :

$$R = \frac{c\, t_{\mathrm{ToF}}}{2}$$

2.4 Jetson Orin Nano

La carte **NVIDIA Jetson Orin Nano** est le calculateur embarqué du Robofleet. Elle appartient à la famille Jetson d'NVIDIA, conçue spécifiquement pour les applications d'intelligence artificielle et de robotique embarquée.



FIGURE 2.6 – NVIDIA Jetson Orin Nano

Elle dispose d'un processeur ARM Cortex-A78AE 6 cœurs, d'un GPU NVIDIA avec 1024 cœurs CUDA et de 4 Go de RAM LPDDR5, ce qui lui confère une puissance

de calcul suffisante pour exécuter simultanément les algorithmes de navigation, de fusion de capteurs et de coordination multi-robots en temps réel.

Ce choix matériel est particulièrement adapté à la robotique pour plusieurs raisons :

— Sa faible consommation électrique (7 à 15 W) permet une autonomie prolongée sur batterie.
— Son GPU intégré accélère les traitements liés à la perception et à la planification de trajectoire.
— Elle est nativement compatible avec ROS 2 et les outils NVIDIA (CUDA, TensorRT) pour l'inférence de modèles d'apprentissage automatique.

Architecture logicielle

Ce chapitre présente les choix techniques effectués pour organiser le logiciel du Robofleet, ainsi que les décisions de conception qui ont guidé notre travail. L'objectif n'est pas de décrire comment exécuter les programmes, mais de comprendre pourquoi ils sont structurés ainsi et comment ils interagissent.

3.1 Choix de l'environnement logiciel

Le système repose entièrement sur **ROS 2 Humble Hawksbill**, le middleware robotique utilisé en recherche et développement. Ce choix, imposé par le constructeur Roboworks, s'est révélé pertinent pour plusieurs raisons.

Contrairement à ROS 1, ROS 2 fonctionne sans nœud maître central : chaque robot peut opérer de manière autonome tout en communiquant avec les autres via le protocole DDS (*Data Distribution Service*). Cette architecture décentralisée est essentielle dans notre cas, car la panne d'un robot ne doit pas bloquer l'ensemble du système.

La communication entre les programmes s'organise autour de deux concepts fondamentaux :

— Les **nodes** — des programmes indépendants, chacun responsable d'une tâche précise (lecture du LiDAR, calcul de trajectoire, contrôle des moteurs). Ils peuvent être démarrés, arrêtés ou remplacés sans affecter les autres.

— Les **topics** — des canaux de communication sur lesquels les nodes publient ou s'abonnent pour échanger des données en temps réel. Par exemple, le node LiDAR publie ses mesures sur le topic `/scan`, que le node de navigation consomme pour planifier la trajectoire.

3.2 Organisation en packages

Le logiciel est organisé en cinq packages ROS 2, chacun ayant un rôle précis et des dépendances bien définies.

Package	Rôle	Dépendances
`turn_on_wheeltec_robot`	Initialisation des processus de base	Aucune
`rosbridge_server`	Pont vers les outils externes	`turn_on_wheeltec_robot`
`nav2_bringup`	Cartographie et navigation	`turn_on_wheeltec_robot`
`wheeltec_multi`	Communication maître-esclave	`turn_on_wheeltec_robot`, `nav2_bringup`
`wheeltec_robot_keyboard`	Contrôle manuel par clavier	`turn_on_wheeltec_robot`

La hiérarchie de dépendances impose un ordre de lancement strict : `turn_on_wheeltec_robot` doit toujours être lancé en premier, car il initialise la communication série avec les moteurs et le filtre de fusion de données, sans lesquels aucun autre programme ne peut fonctionner.

3.3 Description des packages

3.3.1 turn_on_wheeltec_robot — Initialisation du robot

Ce package constitue le point d'entrée de tout le système. Il orchestre quatre sous-programmes lancés simultanément :

- **base_serial** — gère la communication série entre la Jetson Orin Nano et la carte de contrôle des moteurs. C'est par ce canal que passent toutes les commandes de déplacement.
- **wheeltec_ekf** — implémente un filtre de Kalman étendu (EKF) qui fusionne les données d'odométrie et de l'IMU pour obtenir une estimation fiable de la position du robot.
- **joint_state_publisher** — publie l'état des joints du robot, nécessaire pour la visualisation dans Foxglove.
- **robot_mode_description** — charge la description URDF du modèle géométrique du robot, utilisée par les algorithmes de navigation.

Le paramètre `carto_slam` permet de choisir entre deux algorithmes de localisation : AMCL (localisation sur carte existante, utilisé en mode maître-esclave) ou Cartographer (SLAM simultané, utilisé pour créer une carte).

3.3.2 nav2_bringup — Navigation et cartographie

Ce package regroupe trois fonctionnalités :

- **SLAM** (*Simultaneous Localization and Mapping*) — le robot construit une carte de son environnement en temps réel à partir des données du LiDAR, tout en se localisant sur cette carte.
- **Navigation autonome** — la pile Nav2 calcule des trajectoires et pilote le robot vers une destination donnée en évitant les obstacles.

— **Localisation AMCL** (*Adaptive Monte Carlo Localization*) — une fois une carte connue, l'algorithme AMCL positionne le robot sur cette carte à partir des données LiDAR. Cette localisation est indispensable au mode maître-esclave.

3.3.3 wheeltec_multi — Coordination maître-esclave

Ce package contient deux programmes distincts :

— **navigation.launch.py** (maître) — lance la navigation Nav2 du robot maître et active le node `send_tfodom`, qui publie en temps réel la position et la vitesse du maître vers les robots esclaves.

— **wheeltec_slave.launch.py** (esclave) — lance la localisation AMCL, la réception des données du maître, le calcul des commandes de suivi et la gestion de l'évitement d'obstacles.

La transmission des données s'effectue via UDP sur le réseau Wi-Fi local. Les paramètres de cette communication sont modifiables dans le fichier `send_tfodom.py`, notamment la fréquence d'envoi, la qualité de service et l'adresse de destination.

3.3.4 rosbridge_server — Interface avec Foxglove

Ce package crée un pont WebSocket entre ROS 2 et des logiciels externes. Il expose les topics ROS 2 via le protocole `rosbridge`, permettant d'utiliser Foxglove pour visualiser la carte et les données LiDAR.

3.3.5 wheeltec_robot_keyboard — Contrôle clavier

Ce programme permet de contrôler manuellement le robot depuis le clavier. Il publie des messages de type `Twist` sur le topic `/cmd_vel` avec un lissage des commandes pour éviter les à-coups. La touche `b` active le mode omnidirectionnel, exploitant les capacités des roues Mecanum pour des déplacements latéraux.

Réalisation et résultats

4.1 Fonctionnalités réalisées

Cartographie de la salle

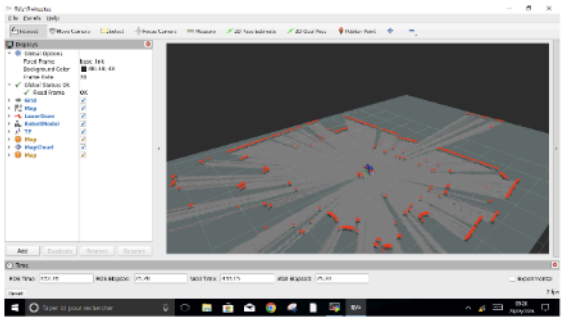


FIGURE 4.1 – Carte

4.2 Difficultés rencontrées et solutions apportées

4.2.1 Optimisation de la communication maître-esclave

Au cours du projet, nous avons été amenés à modifier plusieurs paramètres du fichier `send_tfodom.py` afin d'optimiser la communication entre les robots :

- La **fréquence d'envoi** a été portée de 10 Hz à 20 Hz.
- La **qualité de service** (QoS) a été modifiée pour utiliser `BEST_EFFORT` avec une file de profondeur 1.
- L'**adresse de destination** a été changée de multicast (`224.0.0.7`) vers l'adresse IP directe du robot esclave.

4.2.2 Récupération des fichiers de carte

La commande `scp` étant bloquée sur les robots, nous avons utilisé un serveur web temporaire Python pour transférer les fichiers de carte vers le PC.

Conclusion

Ce projet, centré sur la documentation et l'analyse du fonctionnement d'une flotte de robots, nous a permis de développer une compréhension approfondie de leur architecture et de leurs mécanismes de communication. Le travail en équipe a constitué un véritable atout, favorisant la progression collective et initiant chacun à la démarche de recherche technique, notamment dans l'exploration du comportement maître-esclave des robots.

Cependant, le projet a également mis en lumière plusieurs axes d'amélioration. Sur le plan logiciel, des efforts restent à fournir concernant la partie programmation, que ce soit par l'écriture directe de code ou via l'outil graphique Miros. Du côté matériel et comportemental, des problèmes de mouvements parasites ont été identifiés sur l'unité esclave, et le mode suiveur n'a pas pu être pleinement mis en œuvre, constituant un objectif prioritaire pour la suite.

Enfin, le manque d'aide externe a représenté une contrainte notable tout au long du projet, limitant certaines avancées techniques. Malgré cela, cette expérience a offert un aperçu concret du travail de recherche appliquée, en confrontant l'équipe aux réalités du débogage système et de l'intégration robotique dans un contexte peu documenté.

Bibliographie

[1] Cadden, *Fonctionnement et technologie Lidar*, cadden.fr, 2024.
https://www.cadden.fr/fonctionnement-technologie-lidar/

[2] Redohm, *Roues Mecanum*, redohm.fr, 2016.
https://www.redohm.fr/2016/03/mecanum/

[3] Roboworks, *Multi-Agent ROSbots*, roboworks.net.
https://www.roboworks.net/store/p/multi-agent-rosbots

[4] Zeste de Savoir, *Introduction à LaTeX*, zestedesavoir.com.
https://zestedesavoir.com/tutoriels/826/introduction-a-latex/

[5] NVIDIA, *Jetson Orin Nano — Embedded Systems*, nvidia.com.
https://www.nvidia.com/fr-fr/

[6] Open Robotics, *ROS 2 Humble Hawksbill — Documentation officielle*, docs.ros.org.
https://docs.ros.org/en/humble/index.html

[7] Roboworks, *Training*, roboworks.net.
https://www.roboworks.net/training

[8] Robofleet User Manual , *ROBOWORKS*, roboworks.net.
https://www.roboworks.net/s/Robofleet-User-Manual.pdf